

R.B.GOTHI JAIN COLLEGE FOR WOMEN

PROJECT

AI-powered FAQ and customer support automation

GROUP MEMBERS

THUSHETHA.B
LILLY PUSHPA.K
DHAKSHADHA.M
VASUNTHRA.I
PUNITHA.R

CONTENTS

1. Project Overview
 2. Scenario-Based Case Study
 3. System Requirements
 4. Project Architecture
 5. Database Design
 6. Key Features
 7. Roles and Responsibilities
 8. User Flow
 9. MVC Architecture Pattern
 10. Project Setup and Configuration
 11. Backend Development
 12. Database Configuration
 13. API Router Interface Endpoints
 14. Project Execution and API Testing
 15. Security and Reliability
 16. Conclusion
- Appendix A - Folder Structure
- Appendix B - API Quick Reference

1. Project Overview

1.1 AnswerBot - AI-powered FAQ and customer support automation

The AI FAQ Assistant is a robust, secure, and intelligent REST API platform designed to streamline FAQ creation, user authentication, and AI-powered content management. It is built with Node.js and Express.js and uses MongoDB as its primary data store through Mongoose ODM for data modeling, validation, and database interaction.

Security is a core architectural concern. JSON Web Tokens (JWT) provide stateless authentication and route protection, while bcrypt is used for password hashing. The backend also supports automated FAQ generation through Google Gemini, FAQ search, structured categories, centralized error handling, and standardized JSON responses.

1.2 Project Objectives

- Provide secure user registration and login.
- Maintain reusable FAQ records in MongoDB.
- Allow authorized creators to create, update, and delete FAQ records.
- Allow public consumers to view and search published FAQ content.
- Use Google Gemini to generate AI-assisted FAQ drafts.
- Use Mongoose validation and centralized errors to improve data integrity and reliability.
- Keep routes, controllers, middleware, models, services, and utilities modular.

1.3 Scope

The project is a headless backend service. Standard user-interface template views do not exist inside the backend. Instead, an API routing layer connects HTTP requests to controller logic. This allows Postman, Thunder Client, web applications, mobile applications, or chatbots to consume the same backend services.

2. Scenario-Based Case Study

2.1 Background

Priya is a customer support operations manager handling large backlogs of repetitive user inquiries. She struggles to organize entries, classify topics, write concise responses, and keep support knowledge current while maintaining fast response times.

2.2 Problem

- Difficulty generating and maintaining up-to-date documentation workflows.
- Limited visibility into relevant FAQ/search matches for rapid query resolution.
- Time-consuming manual drafting of repetitive and complex support answers.
- Lack of a unified permission layer protecting private documentation from unauthorized changes.

2.3 Solution

AnswerBot provides a centralized and secure backend API that separates application business logic from frontend delivery systems. Authentication middleware protects private routes, controllers coordinate requests, Mongoose models persist structured data, and an external Gemini service supports automated text generation.

2.4 Usage and Outcome

Usage Type	How the System Is Used
Authenticated Creator	Registers, logs in, requests AI assistance, creates FAQ records, updates or removes owned records, and organizes FAQ content.
Public / Consumer	Views published FAQ information and performs public search operations without modification privileges.
Outcome	Operational delays are reduced; schema validation supports data integrity; security tokens protect modifications; indexed storage supports fast information retrieval.

3. System Requirements

3.1 Software Requirements

Software / Tool	Requirement / Purpose
Operating System	Windows 10/11, macOS, or Linux.
Node.js	Version 18 or above; server runtime.
npm	Version 9 or above; dependency management.
Express.js	RESTful routing and middleware framework.
MongoDB / Mongoose ODM	NoSQL storage, schemas, validation, and references.
Postman / Thunder Client	API verification and protected-route testing.
Visual Studio Code	Source-code editor and integrated terminal.
Google Gemini SDK	External AI service integration for automated FAQ generation.

3.2 Hardware Requirements

Component	Requirement
Processor	Intel Core i5 (8th Gen or above), AMD Ryzen 5, or equivalent.
RAM	8 GB minimum; 16 GB recommended for concurrent MongoDB processing.
Storage	At least 1 GB of available workspace.
Network	Internet access is required when using the external Gemini AI service.

4. Project Architecture

The application exposes structured REST endpoints through a layered server architecture. Requests enter through an Express gateway, pass through middleware, reach controllers, and then interact with data models or external services. This design separates transport, security, domain logic, persistence, and AI integration.

AI FAQ Assistant API – Technical Architecture

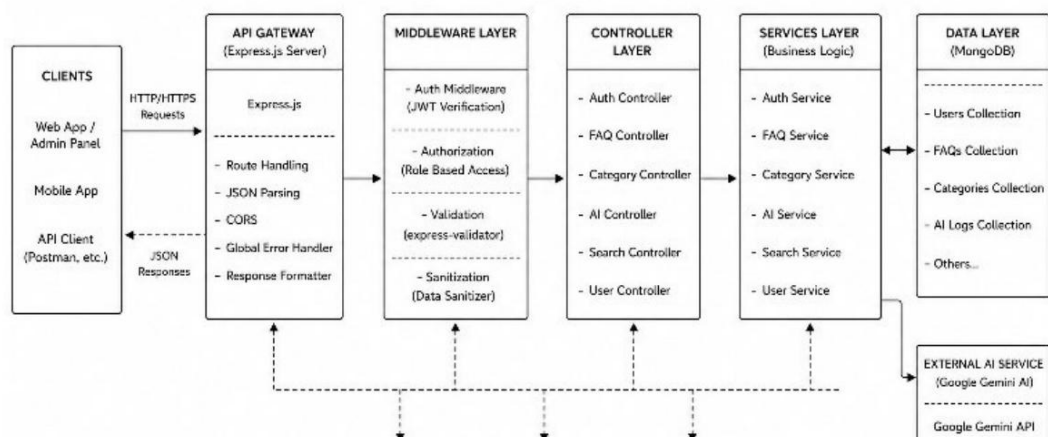


Figure 1. AI FAQ Assistant API - Technical Architecture (from project template)

4.1 Components Definition

Component	Responsibility
Interface / API Client	Postman or another client sends requests and receives JSON entities.
Express Server Gateway	Monitors the network port, maps GET/POST/PUT/DELETE methods, parses JSON, and handles CORS.
Authentication Middleware	Extracts authorization headers, validates token identity, and guards protected resources.
Granular Services	Encapsulate domain logic for sessions, FAQ operations, and external AI integration.
Database Interface Layer	Mongoose translates application operations into MongoDB queries and validates schemas.
AI Flow / Service Integration	Google Gemini SDK performs automated question-answer generation and topic-based inference.
Central Error Handling	Produces standardized failure responses and prevents raw database leakage.

5. Database Design and Entity Relationships

The backend uses MongoDB collections and logical relationships to support user authentication, FAQ management, AI-generated content, and category organization.

Database Interface Layer: Mongoose translates operational application logic into performant native MongoDB queries.

AI Flow (Service Integration): Connected to external AI models via Google Gemini SDK integrations, letting the controller layer offload complex question-answering tasks and automated topic generation workflows.

Entity-Relationship (ER) Diagram Description

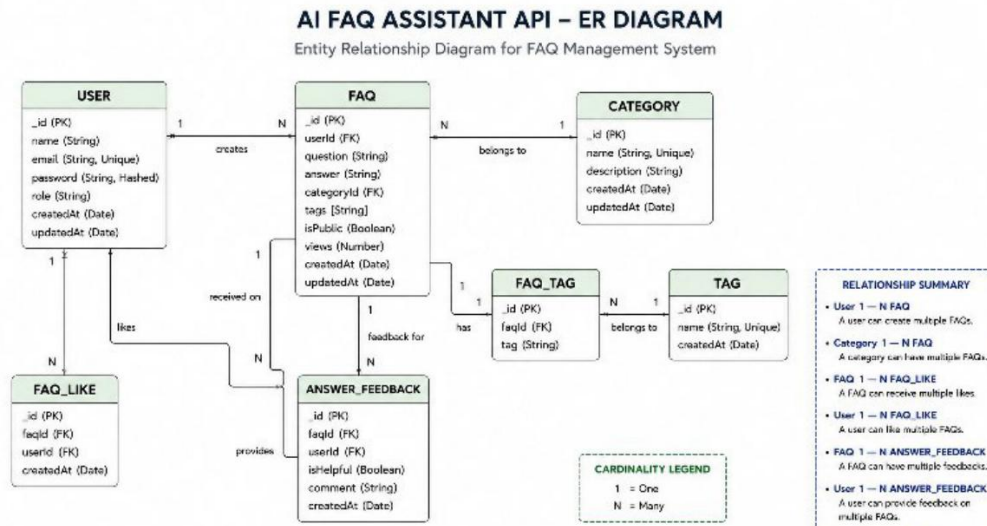


Figure 2. Entity-Relationship Diagram (from project template)

5.1 User Entity

Field	Definition
<code>_id</code>	ObjectId primary key.
<code>name</code>	Required string.
<code>email</code>	Required unique string.
<code>password</code>	Required hashed string.
<code>role</code>	String with default user role.
<code>createdAt</code> / <code>updatedAt</code>	Document timestamps.

5.2 FAQ Entity

Field	Definition
id	ObjectId primary key.
question	Required question string.
answer	Required answer string.
category	Required category string.
createdBy	Foreign key / ObjectId reference to User.
createdAt / updatedAt	Document timestamps.

5.3 Logical Entities and Mappings

AI Generation is produced dynamically through Gemini before persistence and includes topic, generatedQuestion, generatedAnswer, generatedCategory, and generatedAt. Category is described logically using categoryName, description, and totalFAQs.

- User -> FAQ: one-to-many.
- Category -> FAQ: one-to-many.
- User -> AI Generated FAQ: one-to-many.
- FAQ -> Search Results: logical relationship through query matching.

6. Key Features

Feature	Description
JWT Authentication & Authorization	Protects private API endpoints with stateless signed tokens.
AI FAQ Generation	Google Gemini produces FAQ questions, answers, and topic/category suggestions.
FAQ Search	Keyword-oriented and semantic-style retrieval supports faster information discovery.
Complete CRUD	Create, read, update, and delete FAQ records.
Category Management	Organizes FAQ records into structured categories.
Password Encryption	bcrypt hashes passwords before database storage.
Centralized Error Handling	Standardized API error responses improve reliability and debugging.
Schema Validation	Mongoose validates data before persistence.
RESTful API Architecture	Modular Express routes expose scalable backend services.
MongoDB Integration	NoSQL document storage for users and curated FAQ entities.

7. Roles and Responsibilities

Role	Responsibilities
Admin	Manage registered users; view, update, and delete any FAQ; monitor AI-generated content; manage categories; access complete functionality.
Content Creator	Create FAQs; edit and delete owned FAQs; generate FAQ drafts with AI; organize content under categories.
Authenticated User	View public FAQs; search FAQs; generate AI-assisted answers; access personal profile information.
Public User	View published FAQs and search; cannot create, edit, delete, or access protected endpoints.

8. User Flow

The user flow covers registration, login, token issuance, route access, FAQ operations, search, and AI generation. Public paths remain accessible for read/search actions, while modification and AI routes require authentication.

cannot create, edit, or delete FAQ records.

Cannot create, edit, or delete FAQ records.

No access to protected API endpoints.

User Flow

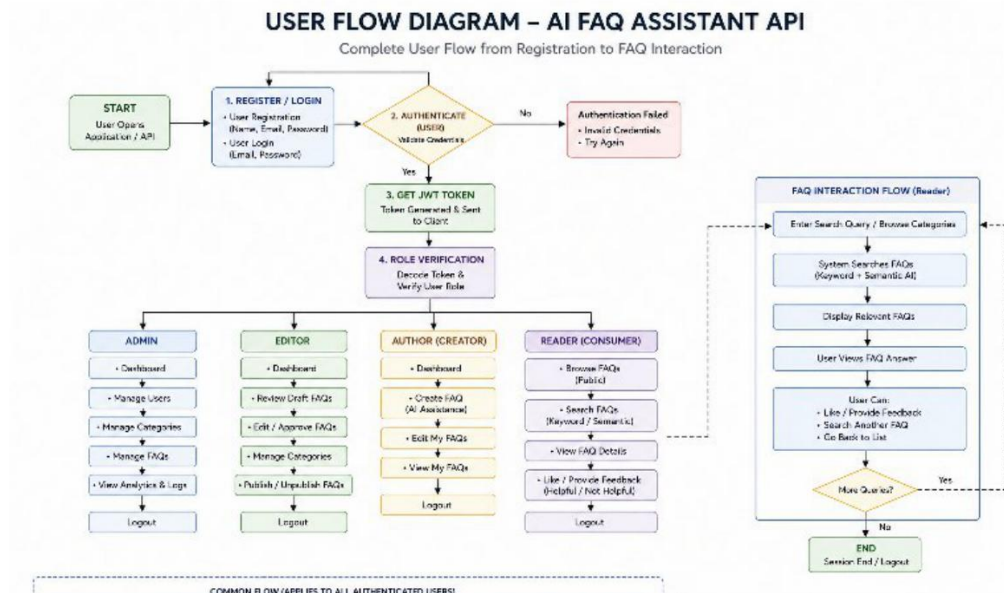


Figure 3. User Flow Diagram (from project template)

8.1 Typical Authenticated Flow

1. Register a new account.
2. Log in with email and password.
3. Receive a JWT token.
4. Send the token in the Authorization header as Bearer <token>.
5. Create, update, or delete authorized FAQ records.
6. Optionally request AI-generated FAQ content and then save approved content.

9. MVC Architecture Pattern

The backend follows a Model-View-Controller style separation. Because it is a headless API, the View role is represented by the routing/network interface rather than server-rendered HTML templates.

The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:

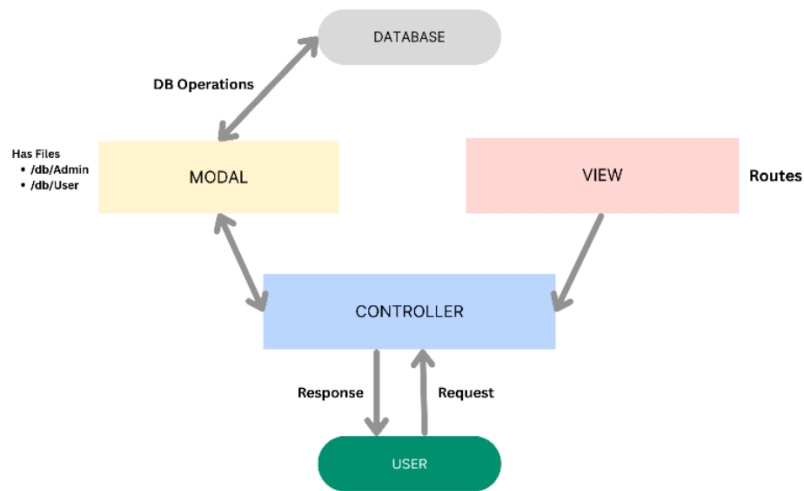


Figure 4. MVC Architecture Pattern (from project template)

9.1 Model Layer

Mongoose schemas define field blueprints, validation constraints, defaults, references, and persistence behavior before data is stored in MongoDB.

9.2 Controller Layer

Controllers capture request context, validate structural constraints, delegate database or service processing, and package results into standard JSON responses.

9.3 View / Routing Interface

Express routes map network requests on specified paths to their corresponding controller functions. This routing layer replaces conventional server-side template views.

10. Project Setup and Configuration

10.1 Project Creation

7. Create a new project folder.
8. Create a src subfolder inside the root.
9. Open the project root in Visual Studio Code.

10.2 npm Initialization and Dependencies

```
npm init -y
npm install express mongoose dotenv cors jsonwebtoken bcrypt @google/genai
```

10.3 Environment Configuration

Environment variables store deployment-specific and sensitive configuration such as PORT, MONGO_URI, JWT_SECRET, JWT expiry, GEMINI_API_KEY, and the configured Gemini model. Secret values should remain private and outside source control.

11. Backend Development

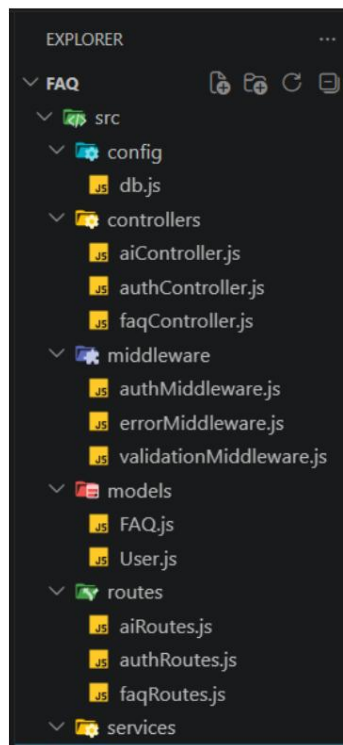


Figure 5. Source Folder Structure (from project template)

File / Module	Purpose
src/server.js	Loads environment settings, starts the database connection, and binds the HTTP listener.
src/app.js	Configures Express, JSON parsing, CORS, routes, and error middleware.
src/middleware/authMiddleware.js	Validates JWT authentication context.
src/middleware/validationMiddleware.js	Rejects invalid request payloads before controller logic.
src/middleware/errorMiddleware.js	Handles errors centrally and sanitizes responses.
src/controllers/authController.js	Handles registration and login.
src/controllers/faqController.js	Coordinates FAQ CRUD and search operations.
src/controllers/aiController.js	Coordinates AI FAQ-generation requests.
src/models/User.js	Defines the user schema.
src/models/FAQ.js	Defines the FAQ schema and creator reference.
src/routes/authRoutes.js	Maps public authentication endpoints.
src/routes/faqRoutes.js	Maps FAQ read/search and protected management endpoints.
src/routes/aiRoutes.js	Maps protected AI automation endpoints.
src/services/geminiService.js	Encapsulates Google Gemini integration.
src/utls/helpers.js	Holds reusable helper functions.

12. Database Configuration

12.1 Mongoose Schema Definitions

The User schema enforces account field validation and password hashing prior to persistence. The FAQ schema models the core FAQ properties and links each record back to its creator through a user reference.

12.2 Data Integrity

Required fields, a unique email constraint, ObjectId references, and timestamp tracking support consistent data storage. Mongoose validation is applied before write operations reach MongoDB.

13. API Router Interface Endpoints

Method	Route	Access	Purpose
POST	/api/auth/register	Public	Register a new profile.
POST	/api/auth/login	Public	Verify user credentials and establish authenticated context.
POST	/api/faqs	Private - Bearer JWT	Create a new FAQ record.
GET	/api/faqs/search?q=configure	Public	Search FAQ records using the q query parameter.
POST	/api/ai/generate-faq	Private - Bearer JWT	Generate an AI-assisted FAQ pair from a supplied topic.

The project also describes complete FAQ CRUD capability. Update and delete operations are protected and subject to creator/authorization rules, while public users do not have modification access.

14. Project Execution and API Testing

14.1 Running the API

Install Postman or Thunder Client for API verification. Start the development server from the project root:

```
npm run dev
```

14.2 Registration Test

Item	Value
Route	POST /api/auth/register
Access	Public

```
{
  "name": "Priya Sharma",
  "email": "priya@writeflow.com",
  "password": "securepassword123"
}
```

14.3 Login Test

Item	Value
Route	POST /api/auth/login
Access	Public

```
{
  "email": "priya@writeflow.com",
  "password": "securepassword123"
}
```

14.4 FAQ Creation Test

Item	Value
Route	POST /api/faqs
Access	Private (Bearer JWT)

```
{
  "question": "How do I configure custom category tags?",
  "answer": "Navigate to settings panel, append keyword tags arrays, and save schema.",
  "category": "Configuration"
}
```

14.5 FAQ Search Test

Item	Value
Route	GET /api/faqs/search?q=configure
Access	Public

14.6 AI FAQ Generation Test

Item	Value
Route	POST /api/ai/generate-faq
Access	Private (Bearer JWT)

```
{
  "topic": "Mongoose schema indexing validation runtime workflow optimization"
}
```

15. Security and Reliability

15.1 JWT Authentication

JWT tokens protect private routes and provide user context without a traditional server-side session.

15.2 Password Hashing

bcrypt hashes passwords before persistence so plaintext passwords are not stored.

15.3 Ownership and Authorization

Content creators manage self-authored records while administrators have broader management responsibilities.

15.4 Schema Validation

Mongoose validates required fields and document structure before database persistence.

15.5 Centralized Error Handling

A common error middleware standardizes status responses and reduces unhandled database detail leakage.

16. Conclusion

AnswerBot demonstrates a modular backend approach to FAQ and customer-support automation. The solution combines Node.js and Express.js routing, MongoDB/Mongoose persistence, JWT authentication, bcrypt password security, FAQ CRUD/search workflows, and Google Gemini-based content generation.

The design directly addresses repetitive support work by centralizing reusable FAQ knowledge and protecting modification operations. Its layered architecture separates routes, middleware, controllers,

services, and data models, enabling future frontend integrations without changing the core backend responsibilities.

Appendix A - Project Folder Structure



Appendix B - API Quick Reference

Operation	Method	Route	Authentication
Register	POST	/api/auth/register	Public
Login	POST	/api/auth/login	Public
Create FAQ	POST	/api/faqs	Bearer JWT
Search FAQ	GET	/api/faqs/search?q=<query>	Public
Generate FAQ	POST	/api/ai/generate-faq	Bearer JWT

Source Basis

This detailed report was prepared from the supplied AI FAQ Assistant project document. It preserves the source document's project terminology, architecture, entity definitions, roles, feature descriptions, software/hardware requirements, development phases, API examples, and testing flow. The embedded architecture, ER, user-flow, MVC, and folder-structure visuals are cropped from the supplied project template for consistency.

Source Codes:

https://drive.google.com/file/d/1QTDxpyKLG_9gjgyNvGLW8DeKj8U0e8BP/view

Demo Video:

<https://drive.google.com/file/d/1kTWQZm0rmaO7Xra1VImVsqBWM-ZsQ0nl/view>